

11. Generation of clock signal using Clocking wizard IP

Aim

To generate multiple clock signals of different frequencies (50 MHz, 25 MHz, and 10 MHz) from a single 100 MHz input clock using the Clocking Wizard IP core of Xilinx Vivado software.

Software Required:

- Xilinx Vivado Design Suite
- Vivado Simulator
- Vivado IP catalog
- PC or Laptop

Procedure

1. Create a New Project:

- Open Xilinx Vivado.
- Create a new RTL project.
- Select the target FPGA device or board.

2. Add Clocking Wizard IP:

- In the IP catalog, search for "**Clocking Wizard**".
- Double-click to add the Clocking Wizard to your design.

3. Configure Input Clock:

- Set the input clock frequency as **100 MHz** (the frequency of your base clock source).

4. Configure Output Clocks:

- Enable 3 output clocks.
- Set the output frequencies as follows:
 - **Output Clock 1:** 50 MHz
 - **Output Clock 2:** 25 MHz
 - **Output Clock 3:** 10 MHz
- Other parameters like phase and duty cycle can be left at default or set as required (usually 0° phase and 50% duty cycle).

5. Generate the Clocking Wizard:

- Click **Generate** to create the IP core with the configured clock outputs.
- The tool will synthesize the MMCM/PLL needed to generate these clocks.

6. Instantiate the IP in Your HDL Code:

- Add the generated Clocking Wizard module in your top-level design.

- Connect the input clock (100 MHz) to the IP's clock input port.
- Use the output clock ports as your new clock signals (50 MHz, 25 MHz, 10 MHz) in your design.

7. Synthesize and Implement the Design:

- Run synthesis and implementation.
- Generate the bitstream for FPGA programming.

8. Verification (Simulation/Hardware):

- Simulate the design to verify clock frequencies using waveform analysis.
- Optionally, program the FPGA and use an oscilloscope or logic analyzer to verify output clock frequencies.

Clocking Wizard Configuration

The image shows the 'Clocking Wizard (6.0)' configuration window. The 'Output Clocks' tab is selected, displaying a table of output clock configurations. The table has columns for Output Clock, Port Name, Output Freq (MHz) Requested, Actual, Phase (degrees) Requested, Actual, and Duty Cycle. The first three rows are checked, showing output clocks at 50 MHz, 25 MHz, and 10 MHz. The remaining four rows are unchecked, showing output clocks at 100 MHz. Below the table, there is a 'USE CLOCK SEQUENCING' checkbox and a 'Clocking Feedback' section with a table for Output Clock, Sequence Number, Source, and Signaling. The 'Source' is set to 'Automatic Control On-Chip' and 'Signaling' is set to 'Single-ended'.

Output Clock	Port Name	Output Freq (MHz) Requested	Actual	Phase (degrees) Requested	Actual	Duty Cycle
☒ clk_out1	clk_out1	50	50.000	0.000	0.000	50.000
☒ clk_out2	clk_out2	25	25.000	0.000	0.000	50.000
☒ clk_out3	clk_out3	10	10.000	0.000	0.000	50.000
☐ clk_out4	clk_out4	100.000	N/A	0.000	N/A	50.000
☐ clk_out5	clk_out5	100.000	N/A	0.000	N/A	50.000
☐ clk_out6	clk_out6	100.000	N/A	0.000	N/A	50.000
☐ clk_out7	clk_out7	100.000	N/A	0.000	N/A	50.000

Testbench program

```
`timescale 1ns / 1ps

module tb_clock_divider_by4;

reg clk;

reg rst;

wire clk_out1;

wire clk_out2;
```

```

wire clk_out3;

clk_wiz_0 uut (
    .clk_out1(clk_out1),
    .clk_out2(clk_out2),
    .clk_out3(clk_out3),
    .reset(rst),
    .clk_in1(clk)
);

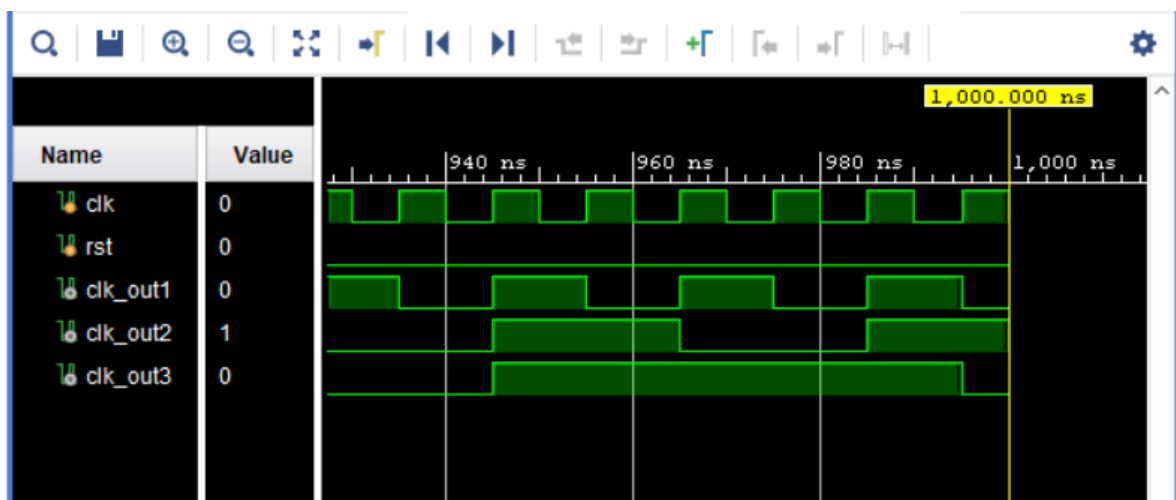
always #5 clk = ~clk;

initial begin
    clk = 0;
    rst = 1;
    #20 rst = 0;
    #100000;
    $stop;
end

endmodule

```

Output



Result

Successfully generated three clock signals with frequencies 50 MHz, 25 MHz, and 10 MHz from a 100 MHz input clock using the Clocking Wizard. The generated clocks were verified through simulation waveforms